

26ICS Lab1 实验报告

汪子斐 24300700002

【实验目的】

本实验完成 Data Lab 的 P1—P19，通过受限运算实现整数与 IEEE 754 单精度浮点数的位级操作，理解补码、掩码、移位、溢出与舍入规则。19 个函数均通过官方规则检查和正确性测试，最终得分为 110/110。

【实验环境与要求】

实验日期为 2026 年 10 月 4 日。使用启智平台的 Ubuntu 22.04、x86-64 环境，按仓库 Makefile 以 GCC 的 -m32、-fwrapv 选项编译。代码位于 wangzifei/DataLab 目录，模板版本为 97132621dc86。

整数题按 32 位补码位向量解释，右移为算术右移，常量限制为 0—255；P2 只允许使用 $\sim$ 和 $\&$ 。浮点题只使用整数和无符号整数，不使用浮点类型、类型转换或函数调用。仅修改 bits.c 中的 19 个函数体，保留函数签名、测试程序、Makefile、规则检查器和评分配置。

【实验步骤】

1. 阅读 bits.c 中每道题的题意、合法运算符和操作次数上限，按位模式推导表达式，再将实现填入对应函数体。
2. 执行 ./check_ops.py bits.c 检查运算规则；编译后执行 ./btest 检查结果。每次修改后重新运行测试，重点检查边界输入。
3. 执行原始 ./test.sh 完成清理、32 位编译、规则检查及正确性检查；另在仓库外进行参考实现对照，并保存日志和测试截图。

【实现思路】

P1 signMask

将常量 1 左移 31 位，得到只有最高位为 1 的位模式 0x80000000。

P2 bitXor

异或要求两个位不同：排除“同时为 1”和“同时为 0”，得到 $\sim(x \& y) \& \sim(\sim x \& \sim y)$ 。表达式只包含 $\sim$ 和 $\&$ 。

P3 negativePart

$x \gg 31$ 将符号扩展为全 0 或全 1； $\sim x + 1$ 构造补码取负。两者按位与，仅保留负数的取负结果。INT_MIN 的取负按题目机器模型保留低 32 位。

P4 copyByteWithin

用 $src \ll 3$ 、 $dst \ll 3$ 将字节编号转换为位偏移。先右移并与 255 提取源字节，再清除目标字节并写入提取值。源字节先从原始 x 取得，因此 $src == dst$ 也成立。

P5 logicalShift

先算术右移，再清除补入的符号位。掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 的低 $32-n$ 位为 1；在 $n = 0$ 时掩码为全 1，避免了移位 32 位的问题。

P6 swapNibblePairs

构造 `0x0f0f0f0f`，分别提取每字节的低半字节和右移后的高半字节，再向相反方向移动 4 位并合并。对算术右移结果加掩码，排除符号扩展。

P7 secondLowestZeroBit

$x | (x + 1)$ 把最低的 0 置为 1，同时恢复加法进位清除的低位 1。设结果为 `filled`，则 $\sim \text{filled} \& (\text{filled} + 1)$ 提取它的最低 0，即原数的第二低 0。少于两个 0 时自然得到 0。

P8 oddParity

依次将相隔 16、8、4、2、1 位的信息异或折叠到最低位，得到原数中 1 的个数的奇偶性。最后用 $!(x \& 1)$ 取反，符合题目“1 的数量为偶数时返回 1”的定义。

P9 rotateRightBits

用 $n \& 31$ 取得实际右旋量；左移量用 $(\sim \text{shift} + 1) \& 31$ 计算，保证总在 0–31 内。将逻辑右移部分与左移部分合并，兼容旋转 0 位和 32 的倍数。

P10 roundEvenPow2

设半个分组为 $\text{half} = 2^{(n-1)}$ ，商的奇偶位为 $(x \gg n) \& 1$ 。给 x 加上 $\text{half} - 1 + \text{parity}$ 后清除低 n 位。非中点时正常就近舍入；恰处中点时，偶数商不进位、奇数商进位。输入上界保证计算不会超出有符号正数范围。

P11 midpointTowardFirst

$(x \& y) + ((x \wedge y) \gg 1)$ 求出向下取整的平均值，不必直接计算可能溢出的 $x+y$ 。若两数奇偶性不同且 $x > y$ ，再加 1，使半整数中点向第一个参数靠拢。比较时先看符号是否不同，仅在同号时使用差值的符号，避免溢出误判。

P12 isBetweenEitherOrder

分别计算有符号关系 $x < a$ 和 $x < b$ ；异号时直接由符号位判断，同号时由差值判断。两个比较结果不同表示 x 落在两个端点之间；再单独纳入 $x == a$ 、 $x == b$ ，实现任意端点顺序的闭区间。

P13 mul5Sat

逐步计算 $2x$ 、 $4x$ 、 $5x$ 。检查这些步骤的符号变化：翻倍时或最后同号相加时出现符号改变，都意味着精确乘积已经超出范围。将溢出标志扩展为全位掩码，再按原数符号选择 `INT_MAX` 或 `INT_MIN`；未溢出时保留 $5x$ 。

P14 classifyAdd3

把每个数精确分解为 $4*(x \gg 2) + (x \& 3)$ 。三个高位部分加上低位和的进位，得到精确总和除以 4 后向下取整的 `high`；其中每步都在 `int` 范围内。原总和能由 `int` 表示，当且仅当 `high` 位于 $[-2^{29}, 2^{29}-1]$ 。检查 $\text{high} \gg 29$ 是否与符号扩展一致，再根据符号返回 1、-1 或 0。也能处理两步相加中先溢出、后相互抵消的情况。

P15 floatScaleThreeHalves

拆出符号、阶码和尾数。规格化数补回隐藏的首位 1，非规格化数保留原尾数；将有效数乘 3 后，根据规格化所需右移 1 或 2 位。使用被丢弃位与半程值比较，按“最近、平局取偶数”一次完成舍入，避免先舍入乘 3 再除以 2 产生二次舍入。

随后处理尾数舍入进位、非规格化数转为规格化数、以及阶码溢出转为无穷大。NaN 原样返回；正负无穷保持原样；正负零保留符号。

P16 floatRoundEven

阶码不小于 150 的数已经没有单精度可表示的小数位，因此直接返回，也包含 NaN 和无穷大。绝对值小于 0.5 时返回带原符号的零；0.5 恰好向偶数 0 舍入，略大于 0.5 则舍入为 1。

其余情况由阶码确定小数位数，清除这些位，并比较余数与半程值。在超过半程或中点且整数部分为奇数时加一个整数单位；编码加法会自然处理阶码进位。

P17 float_i2f

先分离符号，并在 unsigned 中计算绝对值，保证 INT_MIN 也能处理。不断左移，使最高的 1 位于第 31 位，同时调整阶码；保留高 24 位有效数，依据低 8 位执行最近偶数舍入。尾数进位到第 25 位时，相应增加阶码。输入 0 单独返回。

P18 bitCount

使用 0x55555555、0x33333333、0x0f0f0f0f 进行分组累加：先累计每 2 位，再累计每 4 位、每字节，最后合并字节和半字。结果范围是 0-32，保留低 6 位即可。

P19 bitReverse

先分别交换相邻 1 位、2 位和 4 位分组，完成各字节内部反转，再将四个字节倒序排列。为满足 34 次操作限制，先构造 $m4 = 0x0f0f0f0f$ ，通过 $m2 = m4 \wedge (m4 \ll 2)$ 得到 0x33333333，再通过 $m1 = m2 \wedge (m2 \ll 1)$ 得到 0x55555555，减少重复构造掩码的操作。

【实验结果】

```
● [root@DataLab]$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 20/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 21/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 25/65 operators
float_i2f: 23/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1 check_ops.py bits.c 执行结果

```
● [root:DataLab]$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

图 2 btest 执行结果

【实验总结与建议】

本次实验中，符号扩展可将条件转为全位掩码，德摩根律可将逻辑关系转成受限运算，分组累加和逐级交换可处理位计数与位反转。处理比较和中点时，应先分析符号与范围，避免直接求差或求和造成溢出；处理浮点数时，应分别分析规格化、非规格化和特殊值，并在最后统一执行最近偶数舍入。